BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP.HCM
KHOA CÔNG NGHỆ THÔNG TIN

————oOo————



BÁO CÁO ĐỒ ÁN MÔN HỌC

DEEP LEARNING

ĐỀ TÀI:

ỨNG DỤNG KỸ THUẬT TRANSFORMER TRONG BÀI TOÁN PHÁT HIỆN BÌNH LUẬN ĐỘC HẠI

GVHD: PGS.TS. Hoàng Văn Dũng

Học viên thực hiện:

2611318 - **Vòng Vĩnh Shìn**

2611302 - **Trần Nhật Duật**

2611320 - **Hồ Đức Thắng**

__TP. Hồ Chí Minh__, tháng 10 năm 2026

THÔNG TIN NHÓM VÀ TỈ LỆ ĐÓNG GÓP

STT	MSHV	Họ và tên	Tỉ lệ đóng góp (%)	Ghi chú
1	2611318	Vòng Vĩnh Shìn	100%	
2	2611302	Trần Nhật Duật	100%	
3	2611320	Hồ Đức Thắng	100%	

BẢNG PHÂN CÔNG NHIỆM VỤ

[TODO: nhóm chỉnh lại phân công cho đúng thực tế. Dưới đây là bản đề xuất.]

STT	Họ và tên	Nhiệm vụ được phân công	Kết quả
1	Trần Nhật Duật	Tìm hiểu bài toán phát hiện bình luận độc hại, các hướng tiếp cận (luật, học máy truyền thống, RNN và CNN), khảo sát dataset ViHSD và biên soạn phần lý thuyết liên quan.	Hoàn thành
2	Vòng Vĩnh Shìn	Cài đặt khối tiền xử lý dữ liệu (làm sạch, tách từ), xây dựng vocab, mã hoá câu, `Dataset` và `DataLoader`, đồng thời tham gia thực nghiệm.	Hoàn thành
3	Vòng Vĩnh Shìn	Nghiên cứu kiến trúc Transformer (self-attention, multi-head attention, positional encoding, residual connection và LayerNorm), biên soạn phần lý thuyết liên quan.	Hoàn thành
4	Trần Nhật Duật	Cài đặt Transformer Encoder từ đầu bằng PyTorch, vòng lặp huấn luyện với Noam schedule, đánh giá trên tập test, đồng thời phân tích kết quả và lỗi dự đoán.	Hoàn thành
5	Hồ Đức Thắng	Khảo sát ảnh hưởng của learning rate (phát hiện lỗi kết hợp `LambdaLR` với lịch Noam) và vị trí LayerNorm (Post-LN và Pre-LN), chạy thực nghiệm nhiều seed và tổng hợp kết quả (mục 3.4.4).	Hoàn thành
6	Hồ Đức Thắng	Fine-tune mô hình pretrained PhoBERT trên cùng dữ liệu làm mốc tham chiếu, đối chiếu với mô hình tự cài đặt (mục 3.4.5).	Hoàn thành

MỤC LỤC

[Mục lục] Nhấn chuột phải và chọn Update Field để cập nhật.

DANH MỤC BẢNG

[Danh mục bảng] Nhấn chuột phải và chọn Update Field để cập nhật.

DANH MỤC HÌNH ẢNH

[Danh mục hình] Nhấn chuột phải và chọn Update Field để cập nhật.

CHƯƠNG 1: MỞ ĐẦU

1.1. Lý do chọn đề tài

Mạng xã hội như Facebook, YouTube, TikTok đã trở thành không gian giao tiếp chính của hàng chục triệu người dùng Việt Nam. Bên cạnh những trao đổi tích cực, lượng bình luận mang tính xúc phạm, lăng mạ hoặc kích động thù ghét cũng tăng lên nhanh chóng. Các bình luận độc hại này ảnh hưởng tiêu cực đến tâm lý người đọc, làm xấu môi trường thảo luận chung và gây rủi ro pháp lý cho các nền tảng. Với khối lượng dữ liệu khổng lồ được tạo ra mỗi ngày, việc kiểm duyệt thủ công là không khả thi, đòi hỏi phải có các hệ thống tự động phát hiện bình luận độc hại.

Nhóm chúng em lựa chọn đề tài "Ứng dụng kỹ thuật Transformer trong bài toán phát hiện bình luận độc hại" xuất phát từ ba lý do chính sau.

- Thứ nhất, Transformer [1] là kiến trúc nền tảng của hầu hết các mô hình xử lý ngôn ngữ tự nhiên hiện đại như BERT [2], PhoBERT [3] hay các mô hình ngôn ngữ lớn. Việc tự cài đặt một Transformer Encoder hoàn chỉnh, dù ở quy mô nhỏ, là cách hiệu quả để nhóm nắm vững các khái niệm cốt lõi như self-attention, multi-head attention, positional encoding thay vì chỉ sử dụng các thư viện có sẵn như một "hộp đen".
- Thứ hai, bài toán phát hiện bình luận độc hại tiếng Việt có nhiều thách thức thực tế: ngôn ngữ mạng xã hội nhiều từ lóng, viết tắt, sai chính tả, teencode, emoji, đồng thời dữ liệu mất cân bằng lớp nghiêm trọng khi phần lớn bình luận là bình thường. Đây là môi trường tốt để quan sát trung thực khả năng và giới hạn của một mô hình Transformer.
- Thứ ba, nhóm mong muốn làm việc với một bộ dữ liệu thực tế, có gán nhãn và được công bố khoa học là ViHSD [4], để kết quả thực nghiệm có thể tái lập và đối chiếu được.

Xuất phát từ các lý do trên, đề tài của nhóm chúng em tập trung xây dựng và phân tích một mô hình Transformer Encoder được **tự cài đặt từ đầu bằng PyTorch** và **huấn luyện từ trọng số ngẫu nhiên** (không dùng mô hình pretrained) cho bài toán phân loại bình luận thành ba lớp CLEAN, OFFENSIVE và HATE, đồng thời đánh giá

trung thực những gì một Transformer không có tri thức ngôn ngữ tiền huấn luyện có thể đạt được trên dữ liệu tiếng Việt.

1.2. Mục tiêu đề tài

Đề tài hướng đến các mục tiêu cụ thể sau:

- Hệ thống hoá cơ sở lý thuyết về bài toán phát hiện bình luận độc hại và các hướng tiếp cận chính, trọng tâm là kiến trúc Transformer Encoder (embedding, positional encoding, scaled dot-product attention, multi-head attention, feed-forward, residual connection và layer normalization).
- Cài đặt một pipeline hoàn chỉnh từ dữ liệu thô: tải dữ liệu, làm sạch và tách từ tiếng Việt, xây dựng từ điển, mã hoá câu, xây dựng mô hình, huấn luyện, đánh giá và suy luận trên câu mới.
- Tự cài đặt toàn bộ các thành phần của Transformer Encoder bằng các khối cơ bản của PyTorch (`nn.Linear`, `nn.Embedding`, `nn.LayerNorm`, v.v.), không dùng `nn.MultiheadAttention`, `nn.TransformerEncoder` hay thư viện HuggingFace `transformers`.
- Thực nghiệm trên dataset ViHSD, thu thập số liệu định lượng gồm số tham số mô hình, loss, accuracy, precision, recall, F1 theo từng lớp và macro-F1, confusion matrix. Từ đó đánh giá trung thực hạn chế của mô hình huấn luyện từ đầu trên dữ liệu nhỏ và mất cân bằng.

1.3. Đối tượng và phạm vi nghiên cứu

Đối tượng nghiên cứu của đề tài là kiến trúc Transformer Encoder và ứng dụng của nó trong bài toán phân loại văn bản, cụ thể là phát hiện bình luận độc hại tiếng Việt trên mạng xã hội.

- Phạm vi lý thuyết: mô hình hoá bài toán phát hiện bình luận độc hại dưới dạng phân loại văn bản đa lớp, cấu trúc Transformer Encoder theo bài báo gốc *Attention Is All You Need* [1], hàm mất mát cross-entropy có trọng số lớp, bộ tối ưu Adam, lịch học Noam và các độ đo đánh giá cho dữ liệu mất cân bằng.
- Phạm vi thực nghiệm: cài đặt và huấn luyện mô hình trên dataset ViHSD [4] gồm khoảng 33.400 bình luận, giữ nguyên cách chia ba tập train, dev và test

của tác giả. Mô hình dùng từ điển mức từ (word-level) tự xây từ tập train, trọng số khởi tạo ngẫu nhiên.

- Đề tài không triển khai: tiền huấn luyện (pretraining) trên corpus lớn, phần Decoder của Transformer và hệ thống kiểm duyệt trực tuyến. Mô hình pretrained PhoBERT [3] chỉ được fine-tune trên cùng dữ liệu để làm mốc tham chiếu, không phải đối tượng nghiên cứu chính.

1.4. Nội dung thực hiện

- Khảo sát tài liệu, bài báo khoa học về phát hiện ngôn từ thù ghét và xúc phạm, kiến trúc Transformer và dataset ViHSD.
- Xây dựng cơ sở lý thuyết, mô hình hoá bài toán phân loại bình luận ba lớp.
- Thiết kế pipeline xử lý: tiền xử lý văn bản (làm sạch, tách từ), xây dựng từ điển và mã hoá câu, Transformer Encoder, mean pooling và lớp phân loại, huấn luyện và đánh giá.
- Cài đặt bằng Python (PyTorch, underthesea, scikit-learn) trên Jupyter Notebook, thực nghiệm trên ViHSD và thu thập số liệu định lượng.
- Xây dựng ứng dụng web minh hoạ (FastAPI và React) để gắn nhãn bình luận theo thời gian thực và trực quan hoá trọng số attention.
- Phân tích, đánh giá kết quả đạt được, hạn chế và đề xuất hướng phát triển.

1.5. Phương pháp thực hiện

Đề tài của nhóm chúng em kết hợp phương pháp tìm hiểu lý thuyết và phương pháp thực nghiệm xây dựng ứng dụng.

- Về mặt lý thuyết, nhóm tổng hợp và đọc một số công trình, bài báo khoa học đã công bố (chủ yếu là các bài báo open access trên arXiv và tại các hội nghị uy tín về xử lý ngôn ngữ tự nhiên và học máy) để xây dựng nền tảng lý luận nhất quán cho kiến trúc Transformer và bài toán phát hiện bình luận độc hại.
- Về mặt thực nghiệm, nhóm cài đặt trực tiếp từng thành phần của Transformer Encoder bằng PyTorch, huấn luyện trên dataset ViHSD, sau đó thu thập số liệu định lượng và đối chiếu với cơ sở lý thuyết đã trình bày.

1.6. Dự kiến kết quả

Sau khi hoàn thành, đề tài dự kiến đạt được các kết quả sau:

- Báo cáo trình bày có hệ thống cơ sở lý thuyết về bài toán phát hiện bình luận độc hại và kiến trúc Transformer Encoder, có trích dẫn nguồn tham khảo khoa học rõ ràng.
- Một notebook hoàn chỉnh, có thể chạy lại được, cài đặt Transformer Encoder từ đầu, huấn luyện và đánh giá trên ViHSD, kèm hàm suy luận cho câu bình luận mới.
- Một ứng dụng web minh hoạ cho phép nhập bình luận, nhận nhãn dự đoán ngay lập tức, xem các bước tiền xử lý và trọng số attention của từng câu.
- Số liệu kết quả thực nghiệm thực tế mà nhóm thực hiện được (không phải số liệu minh hoạ) bao gồm kích thước từ điển, số tham số mô hình, loss và macro-F1 theo từng epoch, classification report và confusion matrix trên tập test.
- Nhận xét, đánh giá trung thực về ưu điểm và hạn chế của một Transformer huấn luyện từ đầu không có tri thức tiền huấn luyện. Cuối cùng là đề xuất hướng phát triển tiếp theo.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

2.1. Tổng quan về bài toán phát hiện bình luận độc hại

Phát hiện bình luận độc hại (toxic comment detection) là bài toán xác định một đoạn văn bản do người dùng tạo ra trên mạng xã hội có chứa nội dung xúc phạm, lăng mạ hoặc kích động thù ghét hay không [5]. Về mặt hình thức, đây là bài toán phân loại văn bản có giám sát: cho một câu đầu vào $x=(w_1, w_2, \ldots, w_T)$ gồm T từ, mô hình cần học ánh xạ $f: x \mapsto y$ với y thuộc tập nhãn hữu hạn.

Trong đề tài này, nhóm sử dụng dataset ViHSD (Vietnamese Hate Speech Detection) [4] với ba nhãn:

- **CLEAN** (0): bình luận bình thường, không chứa nội dung xúc phạm hay thù ghét.
- **OFFENSIVE** (1): bình luận chứa từ ngữ thô tục, xúc phạm nhưng không nhắm vào một cá nhân hay nhóm cụ thể.
- **HATE** (2): bình luận chứa nội dung lăng mạ, thù ghét nhắm vào một cá nhân hoặc nhóm người cụ thể.

Ranh giới giữa OFFENSIVE và HATE vốn không rõ ràng, ngay cả với người gán nhãn [6]. Kết hợp với đặc thù ngôn ngữ mạng xã hội tiếng Việt (từ lóng, teencode, viết tắt, sai chính tả, emoji, từ được "né" kiểm duyệt bằng cách viết lệch) và phân bố nhãn mất cân bằng nghiêm trọng khi phần lớn bình luận là CLEAN, bài toán này có độ khó cao hơn nhiều so với các bài toán phân loại văn bản chuẩn.

2.2. Phân loại các phương pháp phát hiện bình luận độc hại

2.2.1. Phương pháp dựa trên luật và từ điển

Phương pháp này duy trì một danh sách từ khoá xúc phạm (blacklist) và gán nhãn độc hại cho bình luận chứa các từ đó. Ưu điểm là đơn giản, nhanh, dễ giải thích. Nhược điểm là không hiểu ngữ cảnh (cùng một từ có thể mang nghĩa xúc phạm hoặc không tuỳ câu), dễ bị né tránh bằng cách viết lệch từ và danh sách từ khoá cần cập nhật thủ công liên tục [5].

2.2.2. Phương pháp học máy truyền thống

Văn bản được biểu diễn thành vector đặc trưng thủ công như Bag-of-Words, TF-IDF hay n-gram ký tự, sau đó đưa vào các bộ phân loại như Logistic Regression, SVM, Naive Bayes [6]. Cách tiếp cận này học được từ dữ liệu, bền vững hơn phương pháp luật, nhưng biểu diễn thưa và mất thông tin thứ tự từ, khó nắm bắt ngữ nghĩa và quan hệ xa giữa các từ trong câu.

2.2.3. Phương pháp học sâu tuần tự (RNN, LSTM và CNN)

Các mô hình học sâu học biểu diễn dày (dense embedding) cho từ và tự trích xuất đặc trưng. Mạng hồi quy RNN và biến thể LSTM [7] xử lý câu tuần tự từ trái sang phải, lưu thông tin qua trạng thái ẩn. Mạng tích chập TextCNN [8] dùng các bộ lọc trượt để bắt n-gram cục bộ. Hạn chế chung là RNN không song song hoá được theo chiều thời gian và khó học phụ thuộc xa do gradient tiêu biến, còn CNN chỉ nhìn được ngữ cảnh trong phạm vi cửa sổ bộ lọc.

2.2.4. So sánh mô hình tuần tự và Transformer

Transformer [1] loại bỏ hoàn toàn cơ chế hồi quy, thay bằng cơ chế **self-attention** cho phép mỗi từ tương tác trực tiếp với mọi từ khác trong câu chỉ qua một phép tính. Bảng 1 tóm tắt sự khác biệt giữa hai nhóm mô hình.

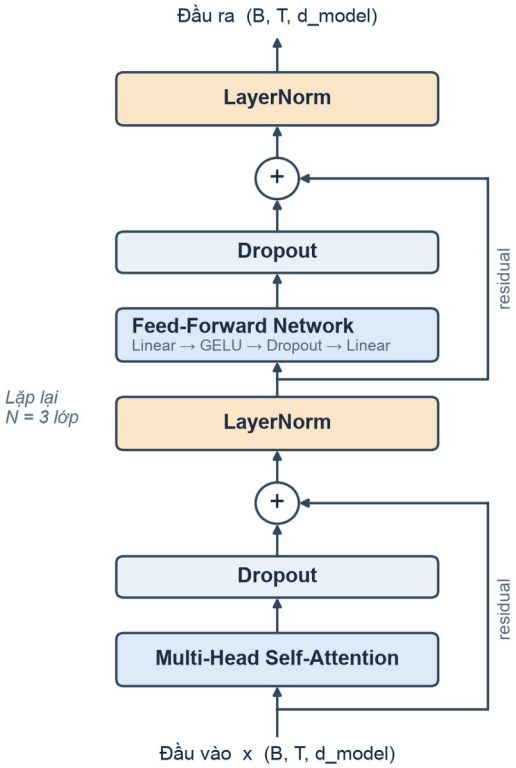
Tiêu chí	**RNN, LSTM**	**Transformer**
Cách xử lý chuỗi	Tuần tự, từng bước thời gian	Song song toàn bộ chuỗi
Độ dài đường đi giữa hai từ bất kỳ	$O(T)$	$O(1)$
Khả năng học phụ thuộc xa	Hạn chế do gradient tiêu biến	Tốt, attention nối trực tiếp mọi cặp từ
Thông tin vị trí	Ngầm định qua thứ tự xử lý	Phải bổ sung tường minh (positional encoding)
Chi phí tính toán theo độ dài câu	$O(T \cdot d^2)$	$O(T^2 \cdot d)$
Yêu cầu dữ liệu	Vừa phải	Lớn, thường cần pretraining để phát huy

Bảng 1: So sánh mô hình tuần tự (RNN, LSTM) và Transformer

Trong phạm vi đề tài này, nhóm lựa chọn tiếp cận theo hướng Transformer Encoder làm nền tảng lý thuyết và thực nghiệm chính. Do bài toán là phân loại (không cần sinh văn bản), chỉ phần Encoder của Transformer được sử dụng, tương tự như cách BERT [2] và PhoBERT [3] được xây dựng.

2.3. Kiến trúc Transformer Encoder

Transformer Encoder gồm một lớp embedding, positional encoding và N lớp encoder giống nhau xếp chồng. Mỗi lớp encoder có hai khối con: Multi-Head Self-Attention và Position-wise Feed-Forward Network, mỗi khối được bọc bởi residual connection và layer normalization (Hình 1).



Hình 1: Kiến trúc một lớp Transformer Encoder (Post-LN)

2.3.1. Token Embedding

Mỗi từ w_t được ánh xạ thành chỉ số nguyên trong từ điển, sau đó tra bảng embedding $E \in \mathbb{R}^{|V| \times d_{model}}$ để thu được vector dày $e_t \in \mathbb{R}^{d_{model}}$. Theo bài báo gốc [1], vector embedding được nhân thêm hệ số $\sqrt{d_{model}}$ để có độ lớn tương đương positional encoding khi cộng lại.

2.3.2. Positional Encoding

Do self-attention không phân biệt thứ tự các từ (hoán vị các từ đầu vào chỉ làm hoán vị đầu ra tương ứng), thông tin vị trí phải được bổ sung tường minh. Transformer gốc dùng positional encoding dạng hình sin [1]:

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right), PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

trong đó pos là vị trí của từ trong câu và i là chỉ số chiều. Mỗi chiều tương ứng một sóng sin với bước sóng từ 2π đến $10000 \cdot 2\pi$, giúp mô hình biểu diễn được cả vị trí tuyệt đối lẫn khoảng cách tương đối giữa các từ, vì PE_{pos+k} có thể biểu diễn bằng một phép biến đổi tuyến tính của PE_{pos}. Positional encoding không có tham số học, được cộng trực tiếp vào embedding: $x_t = \sqrt{d_{model}} \cdot e_t + PE_t$.

2.3.3. Scaled Dot-Product Attention và Multi-Head Attention

Với ma trận đầu vào $X \in \mathbb{R}^{T \times d_{model}}$, self-attention chiếu X thành ba ma trận Query, Key, Value qua các ma trận trọng số học được:

$$Q = XW_Q, K = XW_K, V = XW_V$$

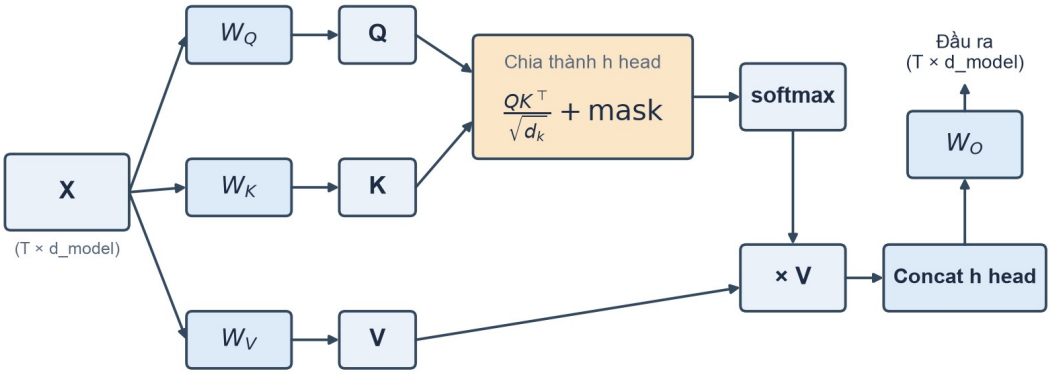
Sau đó tính:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right)V$$

Phần tử (i, j) của $QK^\top$ đo mức độ liên quan giữa từ thứ i (đóng vai trò truy vấn) và từ thứ j (đóng vai trò khoá). Hệ số $\sqrt{d_k}$ giữ phương sai của tích vô hướng ổn định khi d_k lớn, tránh softmax bị bão hoà làm gradient gần bằng 0 [1]. Ma trận mask M nhận giá trị $-\infty$ tại các vị trí đệm `<pad>` và 0 tại các vị trí khác, để sau softmax trọng số chú ý vào phần đệm bằng 0.

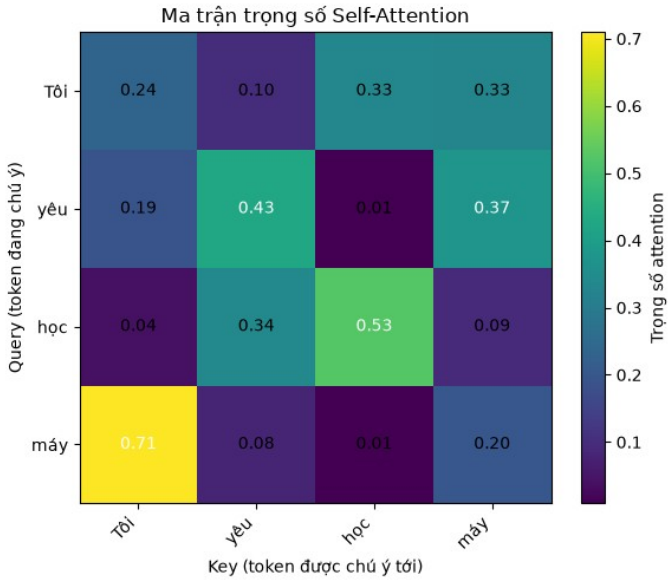
Thay vì một phép attention duy nhất, **Multi-Head Attention** chia không gian d_{model} thành h "đầu" (head) có kích thước $d_k = d_{model}/h$, mỗi head học một kiểu quan hệ khác nhau giữa các từ (ví dụ quan hệ cú pháp, quan hệ đồng tham chiếu, từ phủ định), sau đó ghép kết quả và chiếu lại:

$$\text{MultiHead}(X) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)W_O, \text{head}_i = \text{Attention}(XW_Q^{(i)}, XW_K^{(i)}, XW_V^{(i)})$$



Hình 2: Cơ chế Multi-Head Self-Attention

Để minh hoạ trực quan, nhóm cài đặt self-attention từng bước bằng NumPy trên câu ví dụ "Tôi yêu học máy" (trọng số khởi tạo ngẫu nhiên, chưa huấn luyện). Mỗi hàng của ma trận trọng số là một phân phối xác suất cho biết token truy vấn "chú ý" bao nhiêu đến từng token còn lại, tổng mỗi hàng bằng 1 (Hình 3).



Hình 3: Ma trận trọng số self-attention trên câu ví dụ "Tôi yêu học máy"

2.3.4. Position-wise Feed-Forward Network

Sau attention, mỗi vị trí được đưa độc lập qua một mạng nơ-ron hai tầng với cùng bộ trọng số:

$$\mathrm{FFN}(x) = W_2 \phi(W_1 x + b_1) + b_2$$

với $W_1 \in \mathbb{R}^{d_{model} \times d_{ff}}$, $W_2 \in \mathbb{R}^{d_{ff} \times d_{model}}$, thường chọn $d_{ff} = 4 \cdot d_{model}$. Bài báo gốc dùng hàm kích hoạt ReLU, còn đề tài dùng GELU [9] theo cách của BERT [2]. Nếu

attention đóng vai trò "trộn" thông tin giữa các vị trí, FFN đóng vai trò biến đổi phi tuyến biểu diễn tại từng vị trí.

2.3.5. Residual Connection và Layer Normalization

Mỗi khối con được bọc bởi residual connection [10] và layer normalization [11]. Theo kiến trúc Post-LN của bài báo gốc [1]:

$$x' = \text{LayerNorm}\left(x + \text{Dropout}\left(\text{MultiHead}\left(x\right)\right)\right)$$

$$y = \text{LayerNorm}\left(x' + \text{Dropout}\left(\text{FFN}\left(x'\right)\right)\right)$$

Residual connection giúp gradient truyền trực tiếp qua nhiều lớp, còn LayerNorm chuẩn hoá mỗi vector theo trung bình và độ lệch chuẩn trên chiều đặc trưng, giúp quá trình huấn luyện ổn định. Một biến thể phổ biến là Pre-LN, đặt LayerNorm trước mỗi khối con. Xiong và cộng sự [12] chỉ ra Pre-LN có gradient ổn định hơn ở giai đoạn đầu huấn luyện và ít phụ thuộc vào warm-up, trong khi Post-LN thường cần warm-up learning rate cẩn thận.

2.4. Huấn luyện và đánh giá mô hình phân loại

2.4.1. Hàm mất mát có trọng số lớp

Với đầu ra logits $z \in \mathbb{R}^C$ của C lớp, xác suất dự đoán là $p = \text{softmax}\left(z\right)$. Hàm cross-entropy có trọng số lớp được định nghĩa:

$$L = -\frac{\sum_n w_{y_n} \log p_{n,y_n}}{\sum_n w_{y_n}}, w_c = \frac{N}{C \cdot N_c}$$

trong đó N là tổng số mẫu và N_c là số mẫu của lớp c. Trọng số w_c lớn hơn cho lớp thiểu số giúp mô hình không bị "lười" dự đoán toàn bộ là lớp đa số, vốn là hiện tượng phổ biến trên dữ liệu mất cân bằng như ViHSD.

2.4.2. Bộ tối ưu Adam và lịch học Noam

Transformer thường được huấn luyện bằng Adam [13] với lịch học Noam gồm giai đoạn warm-up tăng tuyến tính rồi giảm theo căn nghịch đảo của số bước [1]:

$$lr\left(step\right) = d_{model}^{-0.5} \cdot min\left(step^{-0.5}, step \cdot warmup^{-1.5}\right)$$

Giai đoạn warm-up tránh cập nhật quá lớn khi trọng số còn ngẫu nhiên và các ước lượng moment của Adam chưa ổn định, đặc biệt quan trọng với kiến trúc Post-LN [12]. Ngoài ra, gradient clipping theo chuẩn [14] được dùng để giới hạn độ lớn gradient, tránh bùng nổ gradient trong các bước đầu.

2.4.3. Các độ đo đánh giá

Với mỗi lớp c, gọi TP_c, FP_c, FN_c lần lượt là số mẫu dự đoán đúng là c, dự đoán nhầm thành c và bỏ sót lớp c:

$$\text{Precision}_c = \frac{TP_c}{TP_c + FP_c}, \text{Recall}_c = \frac{TP_c}{TP_c + FN_c}, F1_c = \frac{2 \cdot \text{Precision}_c \cdot \text{Recall}_c}{\text{Precision}_c + \text{Recall}_c}$$

$$\text{Macro-F1} = \frac{1}{C} \sum_{c=1}^{C} F1_c$$

Trên dữ liệu mất cân bằng, accuracy có thể cao một cách "ảo": một mô hình luôn dự đoán CLEAN vẫn đạt khoảng 83% accuracy trên ViHSD nhưng hoàn toàn vô dụng. Vì vậy, đề tài dùng **macro-F1** làm độ đo chính, do nó coi trọng ngang nhau cả ba lớp, kể cả hai lớp thiểu số OFFENSIVE và HATE. Đây cũng là độ đo được tác giả dataset ViHSD sử dụng [4].

CHƯƠNG 3: GIẢI PHÁP

3.1. Một số kiến trúc tham khảo

Trước khi lựa chọn kiến trúc cài đặt, nhóm đã khảo sát các nhóm kiến trúc phổ biến cho bài toán phát hiện bình luận độc hại, tổng hợp trong Bảng 2. Việc so sánh giúp làm rõ vị trí của giải pháp trong đề tài: một Transformer Encoder tối giản, tự cài đặt và huấn luyện từ đầu, tập trung vào việc hiểu rõ cơ chế bên trong thay vì tối đa hoá độ chính xác.

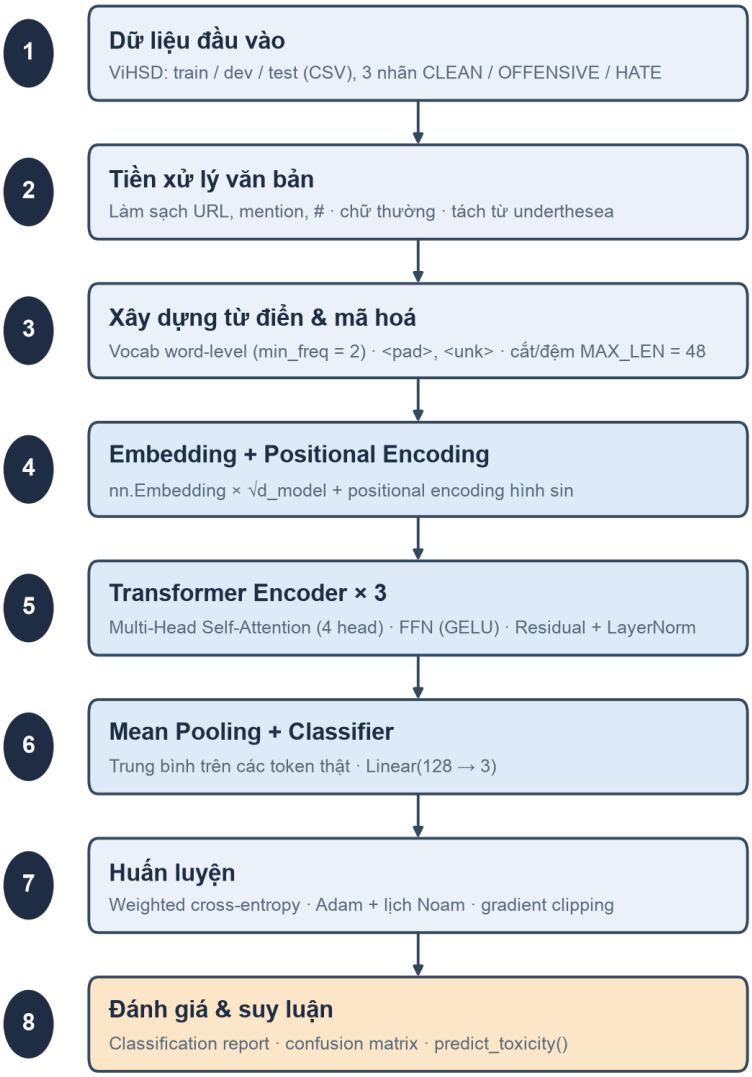
Kiến trúc	**Biểu diễn đầu vào**	**Ưu điểm**	**Nhược điểm**
TF-IDF kết hợp SVM hoặc Logistic Regression	Vector thưa n-gram mức từ hoặc ký tự	Nhanh, ít dữ liệu vẫn chạy được, dễ giải thích	Mất thứ tự từ, không hiểu ngữ cảnh, nhạy với biến thể chính tả
BiLSTM, TextCNN [7], [8]	Word embedding (ngẫu nhiên hoặc pretrained)	Học được đặc trưng tự động, bắt được n-gram và ngữ cảnh cục bộ	RNN khó song song hoá, khó học phụ thuộc xa, còn CNN bị giới hạn bởi cửa sổ lọc
Fine-tune PhoBERT [3]	Subword BPE, Transformer 12 lớp pretrained trên khoảng 20 GB văn bản tiếng Việt	Tận dụng tri thức ngôn ngữ có sẵn, thường cho kết quả tốt nhất	Mô hình lớn (khoảng 135 triệu tham số), cần GPU và bản chất bên trong bị che bởi thư viện
Giải pháp trong đề tài	Từ điển mức từ tự xây từ tập train, embedding khởi tạo ngẫu nhiên	Tự cài đặt toàn bộ Transformer Encoder, minh hoạ trọn vẹn cơ chế attention, mô hình nhỏ gọn (khoảng 1,8 triệu tham số), chạy được trên laptop	Không có tri thức tiền huấn luyện, chỉ học từ khoảng 24 nghìn câu nên độ chính xác thấp hơn đáng kể

Bảng 2: So sánh một số kiến trúc phát hiện bình luận độc hại và giải pháp trong đề tài

Từ bảng so sánh có thể thấy các mô hình mạnh nhất hiện nay đều dựa trên Transformer được tiền huấn luyện trên corpus lớn. Giải pháp trong đề tài chủ động loại bỏ bước tiền huấn luyện và tự cài đặt mọi thành phần, xem đây là bước nền tảng cần nắm vững trước khi sử dụng các mô hình pretrained. Hướng mở rộng cụ thể được thảo luận ở mục 4.3.

3.2. Sơ đồ tổng quát giải pháp

Giải pháp được trình bày dưới dạng một pipeline gồm tám khối xử lý: tải dữ liệu, tiền xử lý văn bản, xây dựng từ điển và mã hoá câu, embedding và positional encoding, Transformer Encoder, mean pooling và phân loại, huấn luyện, đánh giá và suy luận (Hình 4).



Hình 4: Sơ đồ tổng quát pipeline phát hiện bình luận độc hại

3.2.1. Khối tiếp nhận và tiền xử lý dữ liệu

Khối này tải dataset ViHSD từ kho mã nguồn của tác giả [4] (nếu chưa có sẵn) và đọc ba tập train, dev, test. Mỗi bình luận được làm sạch qua các bước: loại bỏ đường dẫn URL, loại bỏ mention dạng `@tên`, loại bỏ ký tự #, gộp khoảng trắng thừa và chuyển về chữ thường.

Do tiếng Việt là ngôn ngữ đơn lập, một từ có nghĩa có thể gồm nhiều âm tiết (ví dụ "việt nam", "lò xo"), văn bản sau làm sạch được tách từ bằng thư viện underthesea [15]. Các âm tiết của cùng một từ ghép được nối bằng dấu _ (ví dụ `việt_nam`, `lò_xo`). Việc tách từ là bước tiền xử lý dữ liệu, không phải một phần của mô hình Transformer. Các câu bị rỗng sau khi làm sạch (ví dụ bình luận chỉ gồm một URL) bị loại bỏ, vì một chuỗi toàn `<pad>` sẽ làm toàn bộ attention mask bằng $-\infty$, gây lỗi NaN khi huấn luyện.

3.2.2. Khối xây dựng từ điển và mã hoá câu

Vì không dùng tokenizer có sẵn, nhóm tự xây từ điển mức từ (word-level) **chỉ từ tập train** để tránh rò rỉ thông tin từ tập dev và test. Các từ xuất hiện ít hơn `MIN_FREQ = 2` lần bị loại để giảm kích thước từ điển và tránh mô hình ghi nhớ các từ chỉ xuất hiện một lần. Hai token đặc biệt được thêm vào: `<pad>` (id = 0) dùng để đệm câu ngắn và `<unk>` (id = 1) thay cho các từ không có trong từ điển.

Mỗi câu được chuyển thành dãy chỉ số, cắt bớt nếu dài hơn `MAX_LEN` và đệm `<pad>` nếu ngắn hơn, tạo thành tensor có kích thước cố định để xử lý theo batch.

3.2.3. Khối Embedding và Positional Encoding

Dãy chỉ số được tra bảng `nn.Embedding` (với `padding_idx = 0` để vector của `<pad>` luôn bằng 0 và không được cập nhật), nhân hệ số $\sqrt{d_{model}}$, sau đó cộng với positional encoding hình sin (mục 2.3.2). Ma trận positional encoding được tính một lần khi khởi tạo và lưu bằng `register_buffer`, nên không phải là tham số học.

3.2.4. Khối Transformer Encoder

Khối này gồm $N = 3$ lớp encoder giống nhau, mỗi lớp được tự cài đặt theo đúng mô tả ở mục 2.3:

- `MultiHeadSelfAttention`: bốn phép chiếu W_Q, W_K, W_V, W_O bằng `nn.Linear`, tensor được tách thành *h* head bằng `view` và `transpose`, điểm attention được chia cho $\sqrt{d_k}$, các vị trí `<pad>` bị gán $-\infty$ bằng `masked_fill` trước softmax.
- `FeedForward`: gồm `Linear(d_model, d_ff)`, hàm kích hoạt GELU, Dropout và `Linear(d_ff, d_model)` nối tiếp nhau.
- `TransformerEncoderLayer`: ghép hai khối trên theo kiến trúc Post-LN, mỗi khối có residual connection, dropout và LayerNorm.

Mask <pad> có kích thước $(B, 1, 1, T)$ để broadcast cho mọi head và mọi vị trí truy vấn.

3.2.5. Khối Mean Pooling và phân loại

Khác với BERT, mô hình không có token `[CLS]` được tiền huấn luyện để đại diện cho cả câu. Vì vậy biểu diễn câu được tạo bằng **mean pooling** trên các vector đầu ra của những token thật (bỏ qua <pad>):

$$h_{\text{câu}} = \frac{\sum_{t=1}^{T} m_t h_t}{\sum_{t=1}^{T} m_t}, m_t = \begin{cases} 1 & \text{nếu token } t \text{ không phải <pad>} \\ 0 & \text{ngược lại} \end{cases}$$

Vector $h_{\text{câu}} \in \mathbb{R}^{d_{model}}$ được đưa qua một lớp `nn.Linear(d_model, 3)` để thu được logits cho ba lớp CLEAN, OFFENSIVE, HATE.

3.2.6. Khối huấn luyện, đánh giá và suy luận

Vòng lặp huấn luyện được viết tay, không dùng `Trainer` của các thư viện bậc cao. Mỗi bước gồm: lan truyền thuận, tính weighted cross-entropy (mục 2.4.1), lan truyền ngược, gradient clipping với chuẩn tối đa 1,0, cập nhật bằng Adam và cập nhật learning rate theo lịch Noam (mục 2.4.2). Sau mỗi epoch, mô hình được đánh giá trên tập dev. Trọng số có **macro-F1 trên dev cao nhất** được giữ lại để đánh giá cuối cùng trên tập test.

Hàm `predict_toxicity()` thực hiện suy luận trên một câu mới qua đúng chuỗi xử lý như lúc huấn luyện (làm sạch, tách từ, mã hoá, đưa qua mô hình và tính softmax), trả về nhãn dự đoán kèm xác suất của từng lớp.

3.3. Thư viện sử dụng và môi trường cài đặt

3.3.1. Thư viện sử dụng

Toàn bộ pipeline được cài đặt bằng Python và chạy trong Jupyter Notebook (`src/vihsd_transformer_from_scratch.ipynb`). Các hàm dùng chung (tải dữ liệu, tiền xử lý, đánh giá) được tách thành package `src/core` để tái sử dụng giữa các notebook. Bảng 3 liệt kê các thư viện chính, phiên bản thực tế trong môi trường thực nghiệm và vai trò của từng thư viện.

Thư viện	Phiên bản	Vai trò trong pipeline
Python	3.13.7	Ngôn ngữ cài đặt toàn bộ pipeline, chạy trong Jupyter Notebook
PyTorch [16]	2.14.0	Chỉ dùng các khối cơ bản (`nn.Linear`, `nn.Embedding`, `nn.LayerNorm`, `nn.Dropout`, `torch.optim`) để tự cài đặt Transformer, tăng tốc bằng Apple MPS
underthesea [15]	9.5.0	Tách từ tiếng Việt (`word_tokenize`) ở bước tiền xử lý
pandas	3.0.6	Đọc CSV, quản lý dữ liệu dạng bảng
NumPy	2.5.3	Tính toán mảng, xử lý kết quả dự đoán
scikit-learn	1.9.1	Tính class weight, accuracy, F1, classification report, confusion matrix
Matplotlib	3.11.2	Vẽ phân bố độ dài câu, đường huấn luyện
seaborn	0.13.2	Vẽ heatmap confusion matrix
tqdm	4.70.1	Hiển thị tiến trình tiền xử lý và huấn luyện
HuggingFace transformers	5.17.0	**Chỉ dùng cho mốc tham chiếu** fine-tune PhoBERT (mục 3.4.5), không dùng trong mô hình chính

Bảng 3: Danh sách thư viện sử dụng

Mô hình chính **không** sử dụng `nn.MultiheadAttention`, `nn.TransformerEncoder`, `nn.TransformerEncoderLayer`, thư viện HuggingFace `transformers` hay bất kỳ embedding hoặc mô hình pretrained nào.

3.3.2. Môi trường cài đặt

Thực nghiệm được chạy trên macOS 26.6.2, chip Apple M1 Pro, RAM 16 GB, sử dụng backend MPS (Metal Performance Shaders) của PyTorch để tăng tốc trên GPU tích hợp. Toàn bộ thư viện được cài trong một môi trường ảo venv riêng, tách khỏi Python hệ thống, để tránh xung đột phiên bản và bảo đảm khả năng tái lập. Các bước thiết lập môi trường:

```
python3 -m venv .venv
source .venv/bin/activate
pip install --upgrade pip
pip install -r requirements.txt
python -m ipykernel install --user --name demo-transformer --
display-name "Python (demo-transformer)"
```

Seed ngẫu nhiên được cố định (`SEED = 42`) cho `random`, NumPy và PyTorch. Tuy nhiên, một số phép tính trên backend MPS không đảm bảo tất định tuyệt đối, nên hai lần chạy cùng cấu hình vẫn có thể lệch nhau nhỏ (xem mục 3.4.4). Thông tin chi tiết của lần chạy thực nghiệm được tổng hợp trong Bảng 5.

3.4. Thực nghiệm

3.4.1. Môi trường và thiết lập thực nghiệm

Nhóm sử dụng dataset ViHSD [4] gồm 33.400 bình luận tiếng Việt thu thập từ Facebook và YouTube, giữ nguyên cách chia ba tập train, dev và test của tác giả. Phân bố nhãn được trình bày trong Bảng 4: lớp CLEAN chiếm khoảng 83%, trong khi OFFENSIVE chỉ chiếm khoảng 7% và HATE khoảng 10%, cho thấy dữ liệu mất cân bằng nghiêm trọng.

Tập	**CLEAN**	**OFFENSIVE**	**HATE**	**Tổng**
Train	19.886 (82,7%)	1.606 (6,7%)	2.556 (10,6%)	24.048
Dev	2.190 (82,0%)	212 (7,9%)	270 (10,1%)	2.672
Test	5.548 (83,1%)	444 (6,6%)	688 (10,3%)	6.680
Tổng	**27.624**	**2.262**	**3.514**	**33.400**

Bảng 4: Phân bố nhãn của dataset ViHSD

Notebook được chạy tuần tự theo đúng thứ tự cell: cài thư viện, tải dữ liệu, tiền xử lý, xây từ điển, tạo `DataLoader`, khai báo kiến trúc, cấu hình mô hình, huấn luyện 15 epoch, đánh giá trên tập test, lưu mô hình và demo suy luận. Bảng 5 tổng hợp các siêu tham số của lần chạy.

Thông số	**Giá trị**
Ngưỡng tần suất từ (MIN_FREQ)	2
Độ dài câu tối đa (MAX_LEN)	48 token
Kích thước từ điển (VOCAB_SIZE)	9.428
Số chiều mô hình (D_MODEL)	128
Số head attention (N_HEADS)	4 ($d_k = 32$)
Số chiều feed-forward (D_FF)	512
Số lớp encoder (N_LAYERS)	3

Thông số	Giá trị
Dropout	0,1
Batch size	32 với tập train, 64 với tập dev và test
Số epoch (EPOCHS)	15
Optimizer	Adam, $\beta = (0,9; 0,98)$, $\epsilon = 10^{-9}$
Lịch học	Noam qua `LambdaLR` với hệ số `lr = 0,2` (LR đỉnh khoảng $5{,}3 \times 10^{-4}$), warm-up 1.128 bước (10% tổng số bước)
Gradient clipping	max norm = 1,0
Hàm mất mát	Cross-entropy có trọng số lớp: 0,4031 (CLEAN), 4,9911 (OFFENSIVE), 3,1360 (HATE)
Tiêu chí chọn mô hình	Macro-F1 cao nhất trên tập dev
Seed	42

Bảng 5: Thông số cấu hình thực nghiệm

3.4.2. Quá trình thực hiện qua từng bước

Phần này bám theo sơ đồ tổng quát ở mục 3.2 và minh hoạ cách chương trình xử lý dữ liệu thực tế qua từng bước.

Bước 1: Tải và đọc dữ liệu.

Hàm `load_vihsd()` kiểm tra thư mục `data/vihsd`. Nếu chưa có dữ liệu, chương trình tải file `vihsd.zip` từ kho mã nguồn của tác giả và giải nén. Ba file `train.csv`, `dev.csv`, `test.csv` được đọc bằng pandas, mỗi file gồm hai cột `free_text` (nội dung bình luận) và `label_id` (nhãn 0, 1 hoặc 2). Số dòng của ba tập lần lượt là 24.048, 2.672 và 6.680.

Bước 2: Tiền xử lý văn bản.

Hàm `preprocess_df()` áp dụng `clean_text()` và `segment_text()` cho từng bình luận. Ví dụ một bình luận trong tập train:

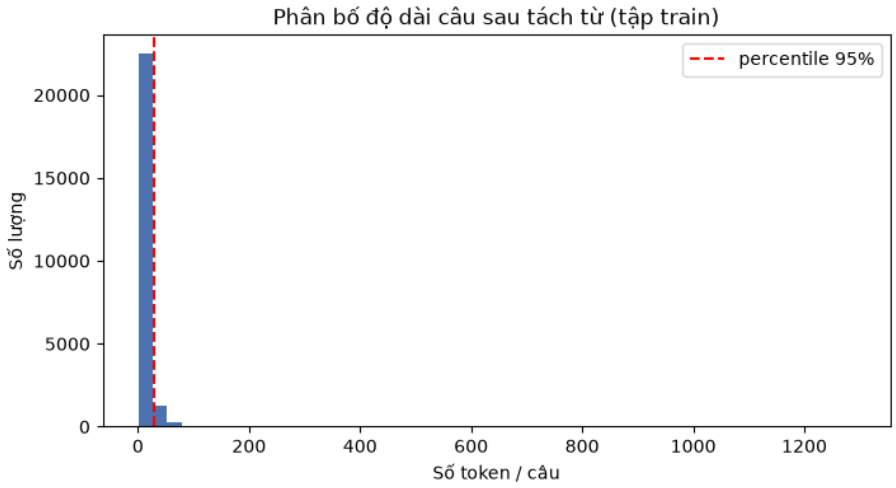
- Gốc: “Đúng là bọn mắt híp lò xo thụt :))) bên việt nam t cái này ra cách đây 10 năm r và bọn t gọi là cái l :)))”
- Sau tiền xử lý: `đúng là bọn mắt híp lò_xo thụt :))) bên việt_nam t cái này ra cách đây 10 năm r và bọn t gọi_là cái l :)))`

Có thể thấy các từ ghép như `lò_xo`, `việt_nam`, `gọi_là` được nhận diện đúng, trong khi các từ viết tắt đặc trưng của mạng xã hội ("t", "r", "l") và emoticon ":)))" được giữ nguyên. Sau bước này có 1 dòng trong tập train bị loại do rỗng sau khi làm sạch, còn tập dev và test không có dòng nào bị loại. Toàn bộ bước tiền xử lý ba tập mất khoảng 36 giây.

Bước 3: Xây dựng từ điển và chọn độ dài câu.

Tập train có 24.448 từ phân biệt. Sau khi loại các từ xuất hiện dưới 2 lần, từ điển còn 9.428 token (gồm <pad> và <unk>). Mười token phổ biến nhất là `.`, `,`, `là`, `có`, `mà`, `cho`, `thì`, `đi`, `này`, `:`, chủ yếu là dấu câu và hư từ.

Để chọn `MAX_LEN`, nhóm khảo sát phân bố độ dài câu sau tách từ trên tập train (Hình 5). Độ dài trung bình là 10,8 token, trung vị 8 token, percentile 95% là 30 token và percentile 99% là 57 token. Phân phối lệch phải mạnh với câu dài nhất lên đến 1.290 token. Giá trị `MAX_LEN = 48` bao phủ khoảng 98,3% số câu. Trên tập test chỉ có 1,7% số câu bị cắt bớt.



Hình 5: Phân bố độ dài câu (số token) sau tách từ trên tập train

Với câu ví dụ ở Bước 2, kết quả mã hoá là dãy 26 chỉ số token thật, tiếp theo là 22 chỉ số `0` (<pad>). Từ ghép `lò_xo` xuất hiện dưới 2 lần trong tập train nên không có trong từ điển và được mã hoá thành `1` (<unk>). Trên tập test, tỉ lệ token bị mã hoá thành <unk> là 8,4%.

Bước 4: Tạo Dataset và DataLoader.

Lớp `ViHSDDataset` mã hoá trước toàn bộ câu thành danh sách chỉ số, phương thức `__getitem__` trả về cặp tensor (input_ids, label). `DataLoader` chia dữ liệu thành

752 batch train (batch size 32, có xáo trộn), 42 batch dev và 105 batch test (batch size 64, không xáo trộn).

Bước 5: Khởi tạo mô hình.

Mô hình `TransformerClassifier` được khởi tạo với cấu hình ở Bảng 5, có tổng cộng **1.801.987 tham số** huấn luyện được, phân bổ như sau:

- Lớp embedding: $9.428 \times 128 = 1.206.784$ tham số (chiếm khoảng 67%).
- Mỗi lớp encoder có 198.272 tham số, gồm multi-head attention 66.048, feed-forward 131.712 và hai LayerNorm 512. Ba lớp encoder có tổng cộng 594.816 tham số.
- Lớp phân loại: $128 \times 3 + 3 = 387$ tham số.

Class weight tính theo công thức ở mục 2.4.1 là 0,4031 cho CLEAN, 4,9911 cho OFFENSIVE và 3,1360 cho HATE, tức một mẫu OFFENSIVE có trọng số gấp khoảng 12 lần một mẫu CLEAN trong hàm mất mát.

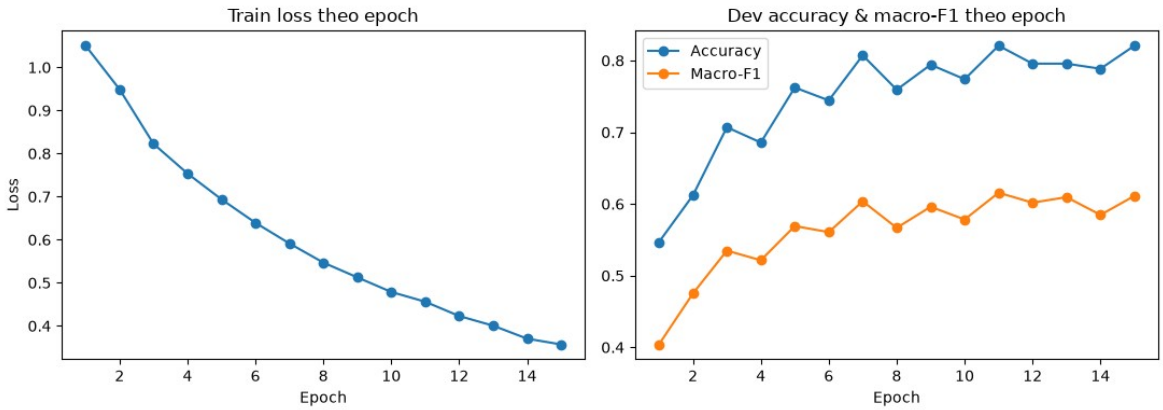
Bước 6: Huấn luyện.

Mô hình được huấn luyện 15 epoch, tổng cộng 11.280 bước cập nhật, trong đó 1.128 bước đầu là warm-up. Với hệ số `lr = 0,2` của `LambdaLR`, learning rate thực tế tăng tuyến tính đến đỉnh khoảng $5,3 \times 10^{-4}$ ở cuối warm-up, sau đó giảm dần còn khoảng $1,7 \times 10^{-4}$ ở bước cuối (cách chọn hệ số này được trình bày ở mục 3.4.4). Toàn bộ quá trình huấn luyện trên Apple M1 Pro (MPS) mất khoảng **3 phút 35 giây**, trung bình khoảng 14 giây mỗi epoch, bao gồm cả đánh giá trên dev. Kết quả từng epoch được ghi lại trong Bảng 6 và Hình 6.

Epoch	Train loss	Dev accuracy	Dev macro-F1
1	1,0502	0,5468	0,4040
2	0,9490	0,6119	0,4750
3	0,8225	0,7070	0,5349
4	0,7536	0,6856	0,5214
5	0,6938	0,7624	0,5691
6	0,6393	0,7444	0,5607
7	0,5914	0,8073	0,6037
8	0,5469	0,7594	0,5667

Epoch	Train loss	Dev accuracy	Dev macro-F1
9	0,5127	0,7942	0,5960
10	0,4788	0,7740	0,5783
11	**0,4561**	**0,8207**	**0,6156**
12	0,4229	0,7957	0,6018
13	0,4005	0,7957	0,6095
14	0,3704	0,7885	0,5847
15	0,3570	0,8207	0,6113

Bảng 6: Kết quả huấn luyện trên tập dev theo epoch



Hình 6: Train loss, accuracy và macro-F1 trên tập dev theo epoch

Train loss giảm đều từ 1,0502 xuống 0,3570. Dev macro-F1 tăng nhanh trong 7 epoch đầu, sau đó dao động trong khoảng từ 0,57 đến 0,62. Trọng số ở epoch 11 (dev macro-F1 = 0,6156) được chọn làm mô hình cuối cùng, sau đó lưu cùng từ điển và cấu hình vào thư mục `models/transformer-scratch-vihsd/` (`model_state_dict.pt`, `vocab.json`, `config.json`).

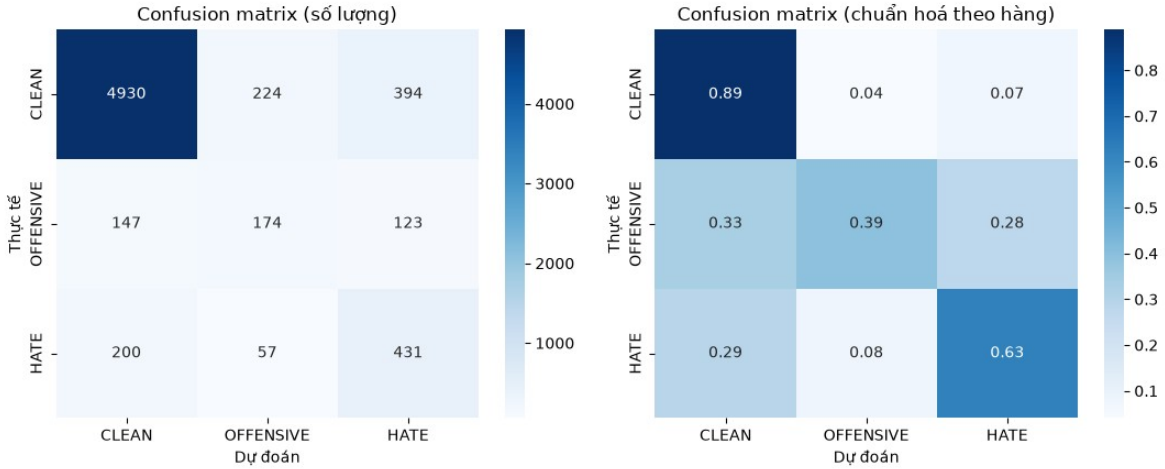
Bước 7: Đánh giá trên tập test và suy luận.

Mô hình tốt nhất được đánh giá trên 6.680 mẫu của tập test, cho kết quả chi tiết ở Bảng 7 và confusion matrix ở Hình 7.

Lớp	Precision	Recall	F1-score	Support
CLEAN	0,9342	0,8886	0,9109	5.548
OFFENSIVE	0,3824	0,3919	0,3871	444
HATE	0,4546	0,6265	0,5269	688
Macro avg	**0,5904**	**0,6357**	**0,6083**	6.680

Lớp	Precision	Recall	F1-score	Support
Weighted avg	0,8482	0,8286	0,8365	6.680
Accuracy			0,8286	6.680

Bảng 7: Kết quả phân loại chi tiết trên tập test



Hình 7: Confusion matrix trên tập test (số lượng và chuẩn hoá theo hàng)

Cuối cùng, hàm `predict_toxicity()` được thử trên ba câu tự viết, kết quả được trình bày trong Bảng 8.

Câu đầu vào	Nhãn dự đoán	P(CLEAN)	P(OFFENSIVE)	P(HATE)
“Video này hay quá, cảm ơn bạn đã chia sẻ nhé”	CLEAN	0,9940	0,0008	0,0051
“Thằng ngu này nói chuyện chán vãi”	HATE	0,0040	0,0031	0,9929
“Bọn mày là lũ súc vật không hơn không kém”	HATE	0,0031	0,0085	0,9884

Bảng 8: Kết quả suy luận trên ba câu tự viết

Mô hình phân loại đúng hướng cả ba câu với độ tự tin cao (xác suất lớp dự đoán trên 0,98). Câu “Thằng ngu này nói chuyện chán vãi” được gán HATE vì nhắm vào một người cụ thể. Tuy nhiên, theo định nghĩa của ViHSD, câu này có thể được gán OFFENSIVE hoặc HATE tuỳ người gán nhãn.

3.4.3. Kết quả thực nghiệm

Bảng 9 tổng hợp các chỉ số định lượng chính của lần chạy thực nghiệm chính thức.

Chỉ số	Giá trị	Ghi chú
Số mẫu tập train	24.047	Đã loại 1 dòng rỗng
Số mẫu tập dev	2.672	
Số mẫu tập test	6.680	
Kích thước từ điển	9.428 token	Từ 24.448 từ phân biệt, MIN_FREQ = 2
Tỉ lệ `<unk>` trên test	8,4%	Token không có trong từ điển train
Tổng số tham số	1.801.987	67% nằm ở lớp embedding
Learning rate đỉnh	khoảng $5{,}3\times10^{-4}$	Noam, hệ số `lr = 0,2`, warm-up 1.128 bước
Thời gian huấn luyện	khoảng 3 phút 35 giây	15 epoch, Apple M1 Pro (MPS)
Train loss (epoch 1 đến 15)	Giảm từ 1,0502 xuống 0,3570	
Dev macro-F1 tốt nhất	0,6156	Epoch 11
Test accuracy	0,8286	1.145 trên 6.680 mẫu dự đoán sai (17,1%)
Test macro-F1	**0,6083**	Độ đo chính
F1 lớp CLEAN (test)	0,9109	
F1 lớp OFFENSIVE (test)	0,3871	
F1 lớp HATE (test)	0,5269	

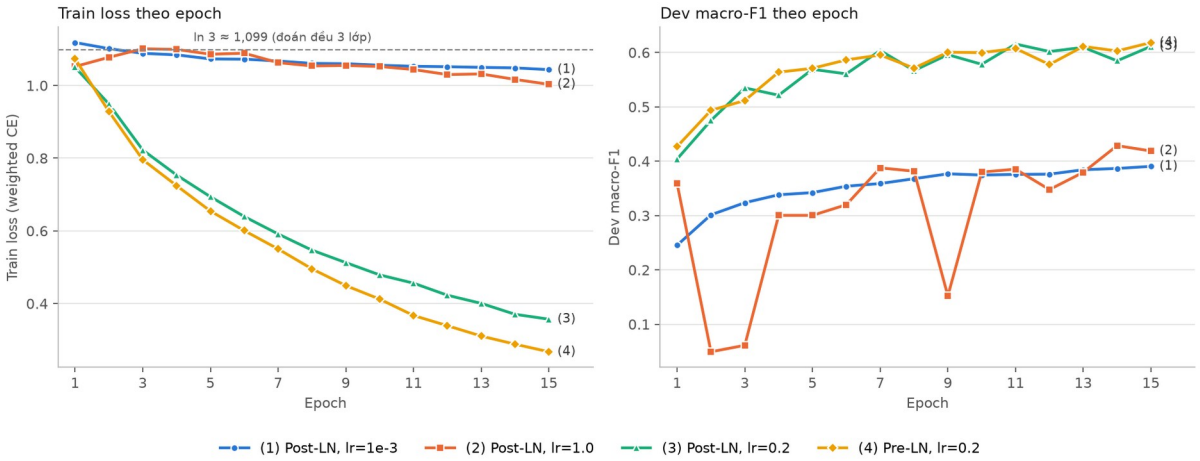
Bảng 9: Kết quả định lượng thực nghiệm

3.4.4. Khảo sát ảnh hưởng của learning rate và vị trí LayerNorm

Trong lần chạy đầu tiên, optimizer được khai báo với `lr = 1e-3`. Do `LambdaLR` **nhân** learning rate ban đầu với giá trị trả về của hàm Noam, trong khi công thức Noam vốn đã là learning rate tuyệt đối, learning rate thực tế chỉ đạt đỉnh $10^{-3}\times128^{-0,5}\times1128^{-0,5}\approx2,6\times10^{-6}$. Mô hình khi đó gần như không học được: train loss dừng ở khoảng 1,04, sát giá trị $\ln3\approx1,099$ của một bộ phân loại đoán đều ba lớp. Nhóm phát hiện lỗi này khi phân tích kết quả và thực hiện thêm các lần chạy với cùng dữ liệu, cùng kiến trúc và cùng seed, chỉ thay đổi hệ số learning rate và vị trí LayerNorm (Bảng 10, Hình 8).

Lần chạy	Kiến trúc	Hệ số `lr`	LR đỉnh	Train loss cuối	Dev macro-F1 tốt nhất (epoch)	Test acc	Test macro -F1	F1 CLEAN	F1 OFF	F1 HATE
(1)	Post-LN	10^{-3}	khoảng $2{,}6\times10^{-6}$	1,0435	0,3905 (15)	0,5683	0,3780	0,716	0,141	0,277
(2)	Post-LN	1,0	khoảng $2{,}6\times10^{-3}$	1,0031	0,4286 (14)	0,6576	0,4061	0,790	0,089	0,340
(3)	**Post-LN**	**0,2**	**khoảng $5{,}3\times10^{-4}$**	**0,3570**	**0,6156 (11)**	**0,8286**	**0,6083**	**0,911**	**0,387**	**0,527**
(3')	Post-LN, chạy lặp lại	0,2	khoảng $5{,}3\times10^{-4}$	0,3624	0,6153 (11)	0,8169	0,5988	0,904	0,354	0,539
(4)	Pre-LN	0,2	khoảng $5{,}3\times10^{-4}$	0,2673	0,6184 (15)	0,8365	0,6045	0,917	0,363	0,533

Bảng 10: Kết quả khảo sát learning rate và vị trí LayerNorm (cùng dữ liệu, cùng seed, 15 epoch)



Hình 8: Train loss và dev macro-F1 theo epoch của bốn cấu hình huấn luyện

Từ Bảng 10 và Hình 8 có thể rút ra các nhận xét sau:

- **Learning rate quá nhỏ (1):** mô hình underfitting nặng, train loss gần như không giảm, dev macro-F1 chỉ tăng chậm lên 0,39.
- **Learning rate quá lớn (2):** với kiến trúc Post-LN, huấn luyện mất ổn định. Có những epoch mô hình "sụp" về dự đoán gần như toàn bộ là lớp thiểu số (dev accuracy chỉ từ 8% đến 12% ở epoch 2, 3 và 9), hoặc toàn bộ là CLEAN (dev accuracy đúng bằng tỉ lệ CLEAN 81,96% ở epoch 4 và 5). Điều này phù hợp với nhận định rằng Post-LN nhạy với learning rate lớn [12].
- **Learning rate phù hợp (3):** hệ số 0,2 cho learning rate đỉnh khoảng $5{,}3\times10^{-4}$, gần với mức đỉnh khoảng 7×10^{-4} của cấu hình gốc trong [1]. Train loss giảm

đều và test macro-F1 tăng từ 0,3780 lên 0,6083. Đây là cấu hình được chọn làm kết quả chính.

- **Pre-LN (4):** cho train loss thấp hơn và đường dev macro-F1 ổn định hơn một chút, nhưng test macro-F1 (0,6045) tương đương Post-LN. Để có Pre-LN, nhóm thêm một LayerNorm cuối (256 tham số) trước bước pooling.
- **Độ dao động giữa các lần chạy:** lần (3') chạy lại đúng cấu hình của lần (3), cùng seed, nhưng test macro-F1 lệch khoảng 0,01 (0,5988 so với 0,6083) do backend MPS không tất định tuyệt đối. Vì chênh lệch giữa Post-LN và Pre-LN ở seed 42 (khoảng 0,004) nhỏ hơn mức dao động này, nhóm chạy thêm hai seed cho mỗi kiến trúc để so sánh đáng tin cậy hơn.

So sánh Post-LN và Pre-LN qua nhiều seed. Với cùng hệ số `lr = 0,2`, mỗi kiến trúc được chạy với ba seed 42, 43 và 44 (seed ảnh hưởng đến khởi tạo trọng số và thứ tự xáo trộn dữ liệu). Kết quả được trình bày dưới dạng trung bình ± độ lệch chuẩn mẫu trong Bảng 11.

Kiến trúc	Dev macro-F1 tốt nhất	Test accuracy	Test macro-F1	F1 CLEAN	F1 OFFENSIVE	F1 HATE
Post-LN	0,6111 ± 0,0049	0,8233 ± 0,0047	**0,6069 ± 0,0080**	0,9075 ± 0,0037	0,3777 ± 0,0180	0,5355 ± 0,0123
Pre-LN	0,6087 ± 0,0098	0,8250 ± 0,0158	**0,6029 ± 0,0072**	0,9093 ± 0,0094	0,3674 ± 0,0082	0,5318 ± 0,0069

Bảng 11: So sánh Post-LN và Pre-LN qua 3 seed (42, 43, 44), `lr = 0,2`, 15 epoch

Với Post-LN, test macro-F1 của seed 42 là 0,6083, seed 43 là 0,6141 và seed 44 là 0,5983. Với Pre-LN, các giá trị tương ứng là 0,6045 (seed 42), 0,6092 (seed 43) và 0,5950 (seed 44). Chênh lệch trung bình giữa hai kiến trúc (0,004) nhỏ hơn độ lệch chuẩn của mỗi kiến trúc (khoảng từ 0,007 đến 0,008) và khoảng giá trị của hai bên chồng lấn nhau. Do đó, trên bộ dữ liệu và quy mô mô hình này, **không có khác biệt đáng kể** giữa Post-LN và Pre-LN khi learning rate đã được chọn phù hợp. Ưu điểm ổn định của Pre-LN [12] chủ yếu thể hiện ở vùng learning rate lớn hoặc mô hình sâu hơn, là những trường hợp đề tài chưa khảo sát.

Về cách chọn cấu hình cuối cùng. Toàn bộ việc chọn cấu hình đều dựa trên **tập dev**, tập test chỉ dùng để báo cáo kết quả:

- Hệ số `lr = 0,2` được chọn vì cho dev macro-F1 cao nhất trong ba mức đã thử (0,6156 so với 0,3905 và 0,4286).
- Post-LN được giữ làm kiến trúc chính vì hai lý do. Thứ nhất, trung bình dev macro-F1 qua ba seed của Post-LN (0,6111) không thấp hơn Pre-LN (0,6087). Thứ hai, Post-LN đúng với kiến trúc gốc trong [1], nhất quán với phần lý thuyết ở Chương 2.
- Trong mỗi lần chạy, trọng số được chọn là epoch có dev macro-F1 cao nhất.

Vì nhóm đã xem kết quả test của nhiều cấu hình trong quá trình khảo sát, con số test của cấu hình được chọn có thể hơi lạc quan. Để giảm rủi ro này, nhóm báo cáo kết quả chính dưới dạng khoảng **0,61 ± 0,01** (trung bình ba seed là 0,6069) thay vì chỉ một con số của một lần chạy.

3.4.5. Đối chiếu với mô hình pretrained PhoBERT

Để có mốc tham chiếu cho kết quả của mô hình tự cài đặt, nhóm fine-tune mô hình pretrained `vinai/phobert-base` [3] trên cùng cách chia dữ liệu, cùng bước tiền xử lý (làm sạch và tách từ bằng underthesea) và cùng class weight. Phần này được cài đặt trong notebook riêng `src/vihsd_phobert_toxic_detection.ipynb`, dùng thư viện HuggingFace `transformers` và không phải trọng tâm của đề tài. Cấu hình fine-tune gồm: tokenizer BPE của PhoBERT với độ dài tối đa 96 subword, 4 epoch, batch size 16, learning rate 2×10^{-5} với 10% warm-up, weight decay 0,01, chọn checkpoint theo dev macro-F1.

Tiêu chí	**Transformer tự cài đặt (Post-LN, 3 seed)**	**PhoBERT fine-tune**
Trọng số ban đầu	Ngẫu nhiên	Pretrained trên khoảng 20 GB văn bản tiếng Việt
Tokenizer	Word-level tự xây (9.428 token)	BPE của PhoBERT (64.001 token)
Số lớp encoder	3	12
Số chiều mô hình d_{model}	128	768
Số tham số	1.801.987	135.000.579 (gấp khoảng 75 lần)
Thời gian huấn luyện (M1 Pro, MPS)	khoảng 3,5 phút (15 epoch)	khoảng 74 phút (4 epoch)

Tiêu chí	**Transformer tự cài đặt (Post-LN, 3 seed)**	**PhoBERT fine-tune**
Dev macro-F1 tốt nhất	0,6111 ± 0,0049	0,6734 (epoch 3)
Test accuracy	0,8233 ± 0,0047	0,8510
Test macro-F1	**0,6069 ± 0,0080**	**0,6539**
F1 lớp CLEAN	0,908	0,927
F1 lớp OFFENSIVE	0,378	0,440
F1 lớp HATE	0,536	0,596

Bảng 12: So sánh Transformer tự cài đặt với PhoBERT fine-tune trên cùng dữ liệu ViHSD

PhoBERT cho test macro-F1 cao hơn khoảng 0,047, vượt trội ở cả ba lớp, rõ nhất ở OFFENSIVE và HATE (mỗi lớp cao hơn khoảng 0,06). Đây là đóng góp của tri thức ngôn ngữ tiền huấn luyện: PhoBERT đã "biết" nghĩa của từ ngữ tiếng Việt trước khi nhìn thấy dữ liệu ViHSD. Tuy nhiên, khoảng cách này nhỏ hơn nhiều so với chênh lệch về quy mô: mô hình tự cài đặt chỉ có khoảng 1/75 số tham số và thời gian huấn luyện ngắn hơn khoảng 20 lần, nhưng vẫn đạt khoảng 93% macro-F1 của PhoBERT. Ngoài ra, lớp OFFENSIVE là lớp yếu nhất ở **cả hai** mô hình (F1 0,38 và 0,44), cho thấy độ khó của lớp này phần lớn nằm ở bản thân dữ liệu, không chỉ ở mô hình.

3.5. Phân tích và đánh giá kết quả

Kết quả tích cực nhất là pipeline chạy hoàn chỉnh từ dữ liệu thô đến suy luận trên câu mới, với toàn bộ thành phần Transformer tự cài đặt. Sau khi điều chỉnh learning rate, mô hình đạt **test macro-F1 = 0,6083** và accuracy 0,8286 ở lần chạy chính thức, trung bình ba seed là 0,6069 ± 0,0080. Kết quả này vượt xa các mốc tham chiếu đơn giản: luôn dự đoán CLEAN cho macro-F1 khoảng 0,30, còn đoán ngẫu nhiên đều ba lớp cho khoảng 0,25. Mô hình phát hiện được 62,7% số bình luận HATE (431 trên 688 mẫu) và các câu demo được phân loại đúng với độ tự tin cao.

Tuy nhiên, kết quả cũng bộc lộ nhiều hạn chế:

- **Lớp OFFENSIVE vẫn khó phân biệt nhất.** F1 của OFFENSIVE là 0,3871. Trong 444 mẫu OFFENSIVE chỉ có 174 mẫu (39,2%) được dự đoán đúng, 147 mẫu (33,1%) bị nhầm thành CLEAN và 123 mẫu (27,7%) bị nhầm thành HATE. Điều này phản ánh cả việc lớp này có ít mẫu nhất (1.606 mẫu train), lẫn ranh giới vốn mơ hồ giữa OFFENSIVE và HATE [6]. Ví dụ câu "Đánh chết mẹ

thằng này đi" được gán nhãn OFFENSIVE nhưng mô hình dự đoán HATE. Đây là trường hợp mà chính người gán nhãn cũng có thể không thống nhất.

- **Precision của HATE còn thấp (0,4546).** Có 394 trên 5.548 bình luận CLEAN (7,1%) bị dự đoán là HATE. Đây là đánh đổi của class weight cao cho lớp thiểu số: mô hình ưu tiên "báo động nhầm" hơn là bỏ sót. Nhiều câu bị nhầm là câu ngắn, thiếu ngữ cảnh hoặc chứa từ lóng như "Lú TBT", "Cái này thầy live lúc nài zay tụi bay".
- **Bắt đầu overfitting.** Từ epoch 7 trở đi, train loss vẫn giảm (từ 0,59 xuống 0,36) nhưng dev macro-F1 không tăng thêm mà dao động trong khoảng từ 0,57 đến 0,62. Với khoảng 1,8 triệu tham số, trong đó 1,2 triệu nằm ở lớp embedding, mô hình dễ ghi nhớ tập train nhỏ.
- **Nhạy với siêu tham số tối ưu và không tất định.** Mục 3.4.4 cho thấy chỉ riêng hệ số learning rate đã làm test macro-F1 thay đổi từ 0,38 đến 0,61. Ngoài ra, kết quả chênh lệch khoảng 0,01 giữa các seed và giữa các lần chạy cùng seed, nên chỉ những khác biệt lớn hơn mức này mới có ý nghĩa. Ví dụ, khác biệt giữa Post-LN và Pre-LN (0,004) là không có ý nghĩa.
- **Không có tri thức ngôn ngữ tiền huấn luyện.** Embedding khởi tạo ngẫu nhiên và chỉ học từ khoảng 24 nghìn câu, trong đó phần lớn là CLEAN. Mô hình phải tự suy ra nghĩa xúc phạm của từ ngữ từ số ít mẫu độc hại. So với PhoBERT (mục 3.4.5), mô hình thấp hơn khoảng 0,047 macro-F1, rõ nhất ở hai lớp thiểu số.
- **Hạn chế của từ điển mức từ.** Có 8,4% token trên tập test bị mã hoá thành `<unk>`, nên một phần tín hiệu có thể bị mất. Ngôn ngữ mạng xã hội còn có nhiều biến thể của cùng một từ. Trong tập train, "nài" chỉ xuất hiện 4 lần so với 1.804 lần của "này", "zay" 8 lần so với 629 lần của "vậy", "wa" 41 lần so với 1.300 lần của "quá". Từ điển mức từ coi chúng là các token hoàn toàn khác nhau, nên embedding của các biến thể hiếm được học rất kém.

Tóm lại, một Transformer Encoder nhỏ, tự cài đặt và huấn luyện từ đầu có thể đạt macro-F1 khoảng 0,61 ± 0,01 trên ViHSD sau khoảng 3,5 phút huấn luyện trên laptop, tương đương khoảng 93% kết quả của PhoBERT fine-tune dù chỉ có khoảng 1/75 số tham số. Tuy nhiên, điều này chỉ đạt được khi learning rate được thiết lập đúng. Hai lớp thiểu số, đặc biệt là OFFENSIVE, vẫn là điểm yếu chính. Đây là những

vấn đề mà các hướng phát triển ở mục 4.3 (tiền huấn luyện, tokenizer subword, xử lý mất cân bằng) hướng tới giải quyết.

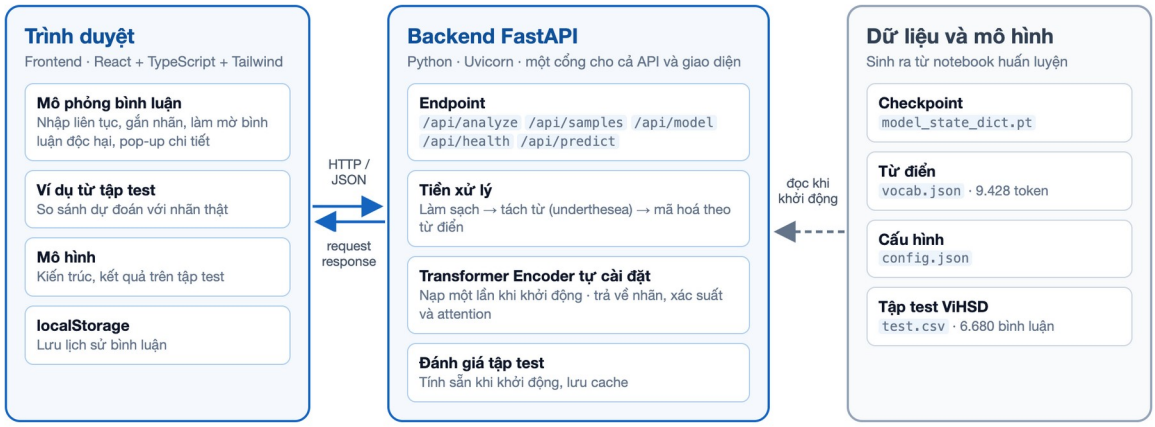
3.6. Xây dựng ứng dụng minh hoạ

Để đưa mô hình ra khỏi notebook và quan sát cách nó hoạt động trong một tình huống gần với thực tế, nhóm xây dựng một ứng dụng web minh hoạ. Ứng dụng mô phỏng khung bình luận của mạng xã hội: người dùng nhập bình luận, mô hình gắn nhãn ngay lập tức, bình luận độc hại được làm mờ giống một hệ thống kiểm duyệt. Với mỗi bình luận, người dùng có thể xem lại toàn bộ quá trình xử lý, từ câu sau làm sạch, câu sau tách từ đến trọng số attention của từng layer và head.

3.6.1. Kiến trúc hệ thống

Ứng dụng gồm ba thành phần, được minh hoạ trong Hình 9:

- **Backend FastAPI.** Khi khởi động, server nạp checkpoint, từ điển và cấu hình do notebook huấn luyện lưu ra (mục 3.4.2, Bước 7), đồng thời chạy mô hình trên toàn bộ tập test một lần và lưu kết quả vào bộ nhớ đệm. Server dùng lại đúng các hàm làm sạch và tách từ của notebook, nên câu được xử lý giống hệt lúc huấn luyện. Các endpoint chính gồm `/api/analyze` (phân tích một câu, trả về nhãn, xác suất, token và ma trận attention của mọi layer và head), `/api/samples` (lấy ngẫu nhiên câu từ tập test) và `/api/model` (cấu hình và kết quả trên tập test). FastAPI tự sinh tài liệu API tại địa chỉ `/docs`.
- **Giao diện web.** Viết bằng React, TypeScript và Tailwind CSS, giao tiếp với backend qua HTTP và JSON. Giao diện được build thành các file tĩnh và do chính FastAPI phục vụ trên cùng một cổng, nên toàn bộ ứng dụng chạy bằng một lệnh `make start`.
- **Dữ liệu và mô hình.** Gồm checkpoint, từ điển, file cấu hình và tập test ViHSD. Ứng dụng không huấn luyện lại mô hình mà chỉ đọc các file này.



Hình 9: Kiến trúc ứng dụng minh hoạ

Khi suy luận, ứng dụng chạy mô hình trên CPU. Do backend MPS không tất định (mục 3.4.4), kết quả trên tập test mà ứng dụng tính lại (accuracy 0,8308 và macro-F1 0,6105) lệch nhẹ so với lần chạy chính thức ở Bảng 9 (0,8286 và 0,6083).

3.6.2. Các chức năng chính

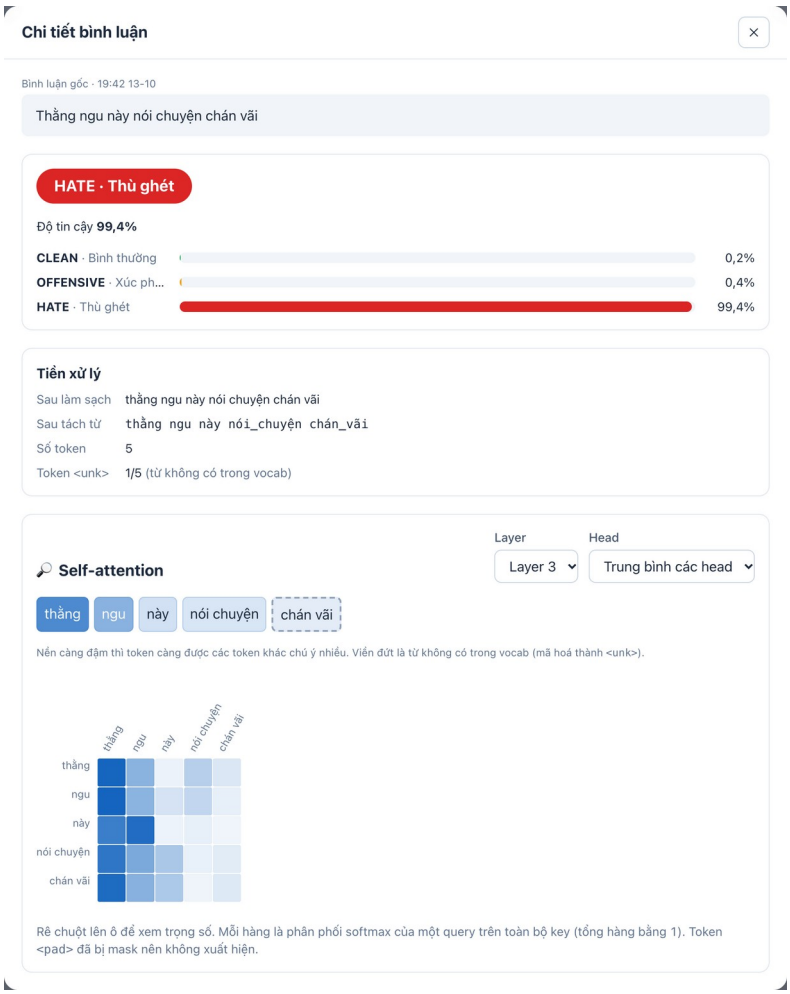
Mô phỏng bình luận. Đây là màn hình chính của ứng dụng (Hình 10). Người dùng nhập bình luận và bấm Enter để gửi. Có thể gửi liên tục mà không cần chờ kết quả của bình luận trước. Mỗi bình luận được gắn nhãn CLEAN, OFFENSIVE hoặc HATE kèm độ tin cậy, viền bên trái tô theo màu của nhãn. Khi bật chế độ **Ẩn bình luận độc hại**, các bình luận OFFENSIVE và HATE bị làm mờ kèm cảnh báo, người dùng bấm **Xem** để hiện nội dung. Câu chỉ chứa đường dẫn hoặc mention sẽ rỗng sau khi làm sạch và được gắn nhãn TRỐNG. Phía trên khung bình luận có thống kê số bình luận theo từng nhãn. Lịch sử bình luận được lưu trong trình duyệt nên không mất khi tải lại trang.



Hình 10: Màn hình mô phỏng bình luận, hai bình luận HATE đang bị làm mờ

Các ví dụ trong Hình 10 cho thấy mô hình phân biệt được một số trường hợp tinh tế. Câu **"Bài này viết dở quá, đọc mà buồn ngủ"** mang ý chê nhưng không chứa từ ngữ xúc phạm nên được gán CLEAN. Câu **"Mẹ nó chứ, lại hết hàng rồi"** có từ ngữ thô tục nhưng không nhắm vào ai nên được gán OFFENSIVE, đúng với định nghĩa của ViHSD (mục 2.1). Câu **"Thằng ngu này nói chuyện chán vãi"** nhắm vào một người cụ thể nên được gán HATE.

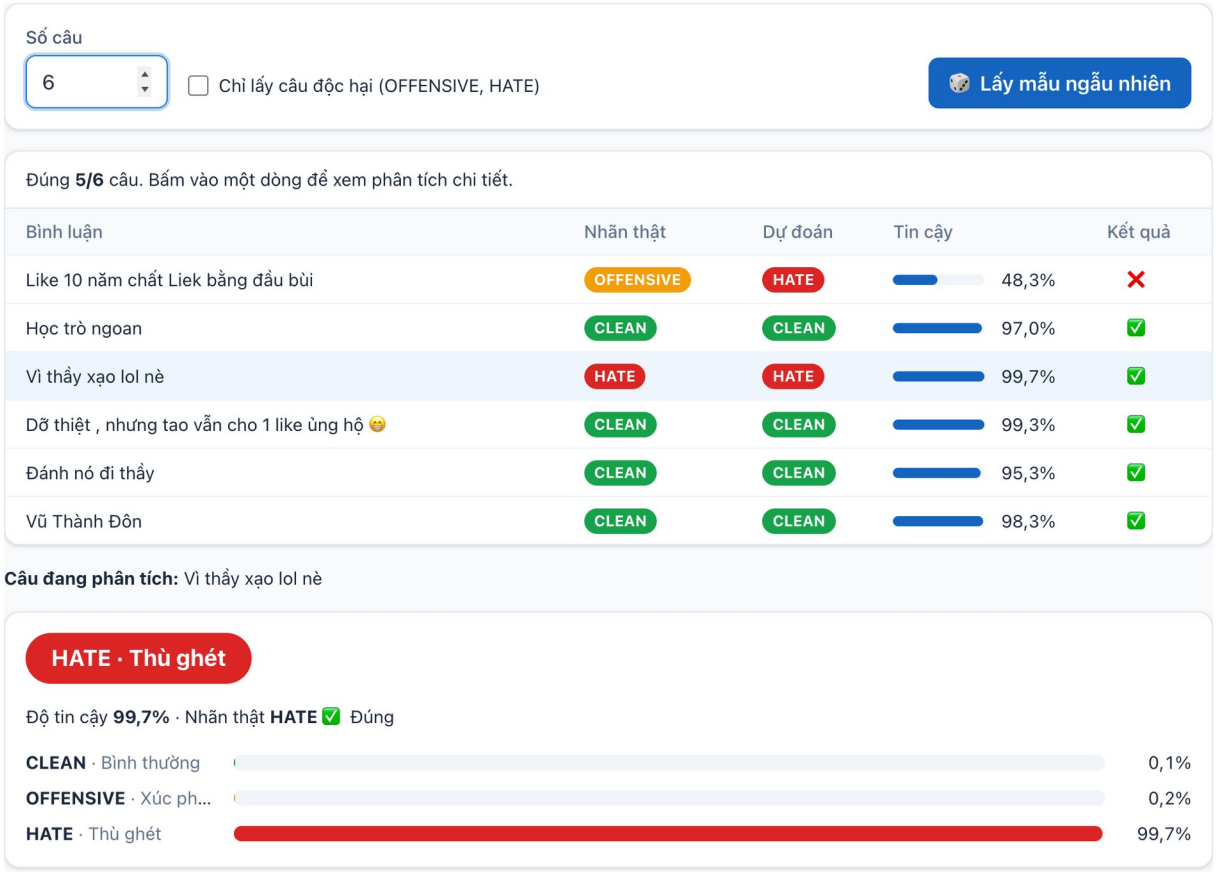
Chi tiết bình luận. Khi bấm vào một bình luận, ứng dụng mở cửa sổ chi tiết (Hình 11) gồm bốn phần: bình luận gốc, nhãn dự đoán kèm xác suất của ba lớp, các bước tiền xử lý (câu sau làm sạch, câu sau tách từ, số token, số token <unk>) và trọng số self-attention. Ở phần attention, người dùng chọn layer và head (hoặc trung bình các head). Mỗi token được tô nền theo mức attention mà nó nhận được từ các token khác. Token không có trong từ điển được đánh dấu bằng viền đứt. Heatmap bên dưới thể hiện đầy đủ ma trận trọng số, mỗi hàng là phân phối softmax của một query trên toàn bộ key.



Hình 11: Cửa sổ chi tiết của một bình luận được gán nhãn HATE

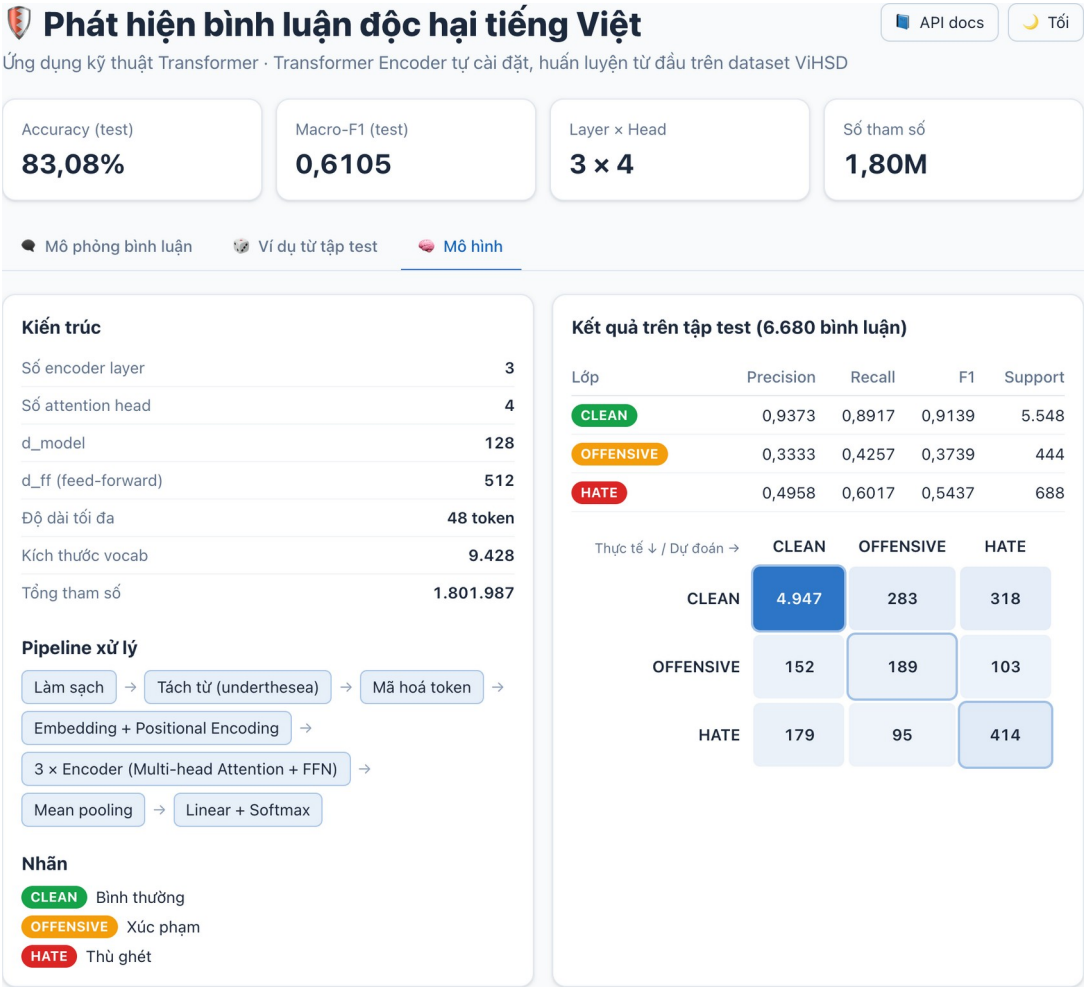
Trong ví dụ ở Hình 11, từ ghép `chán_vãi` không có trong từ điển nên được mã hoá thành `<unk>`. Ở layer cuối, phần lớn trọng số attention tập trung vào hai token "thằng" và "ngu", là những từ mang sắc thái xúc phạm trong câu. Cần lưu ý trọng số attention chỉ cho biết mô hình tổng hợp thông tin từ đâu, chưa đủ để kết luận đó là nguyên nhân của quyết định phân loại.

Ví dụ từ tập test. Màn hình này lấy ngẫu nhiên một số câu từ tập test, có tuỳ chọn chỉ lấy câu độc hại, rồi hiển thị nhãn thật, nhãn dự đoán và độ tin cậy của từng câu (Hình 12). Bấm vào một dòng sẽ hiện phân tích chi tiết của câu đó, giống cửa sổ ở Hình 11. Trong ví dụ ở Hình 12, mô hình dự đoán đúng 5 trên 6 câu. Câu sai duy nhất có nhãn thật OFFENSIVE nhưng bị dự đoán thành HATE với độ tin cậy chỉ 48,3%, minh hoạ đúng điểm yếu đã phân tích ở mục 3.5.



Hình 12: Màn hình ví dụ từ tập test

Thông tin mô hình. Màn hình cuối tóm tắt kiến trúc mô hình, pipeline xử lý và kết quả trên tập test gồm precision, recall, F1 của từng lớp cùng confusion matrix (Hình 13). Các số liệu này được tính trực tiếp từ checkpoint mà ứng dụng đang dùng.



Hình 13: Màn hình thông tin mô hình

CHƯƠNG 4: KẾT LUẬN

4.1. Kết luận

Qua đề tài, nhóm đã tìm hiểu bài toán phát hiện bình luận độc hại, các hướng tiếp cận chính và đặc biệt là các thành phần cốt lõi của kiến trúc Transformer Encoder gồm token embedding, positional encoding hình sin, scaled dot-product attention, multi-head attention, feed-forward network, residual connection và layer normalization. Từ phần lý thuyết này, nhóm tự cài đặt được toàn bộ Transformer Encoder bằng các khối cơ bản của PyTorch, không dùng các lớp Transformer có sẵn hay mô hình pretrained. Nhóm cũng xây dựng một pipeline hoàn chỉnh: tiền xử lý và tách từ tiếng Việt, xây dựng từ điển, huấn luyện với weighted cross-entropy và lịch học Noam, đánh giá và suy luận trên câu mới.

Trong lần chạy thực nghiệm chính thức trên dataset ViHSD, mô hình có 1.801.987 tham số, được huấn luyện 15 epoch trong khoảng 3 phút 35 giây trên Apple M1 Pro. Mô hình đạt macro-F1 0,6156 trên tập dev, **accuracy 0,8286 và macro-F1 0,6083 trên tập test**, phát hiện được 62,7% số bình luận HATE. Qua ba seed, test macro-F1 trung bình là 0,6069 ± 0,0080. Làm mốc tham chiếu, PhoBERT fine-tune trên cùng dữ liệu đạt test macro-F1 0,6539, cao hơn khoảng 0,047 nhưng lớn gấp khoảng 75 lần và cần thời gian huấn luyện gấp khoảng 20 lần.

Một kết quả quan trọng của đề tài là khảo sát ảnh hưởng của learning rate. Lần chạy ban đầu có lỗi trong cách kết hợp `LambdaLR` với lịch Noam, khiến learning rate thực tế chỉ khoảng $2,6 \times 10^{-6}$ và macro-F1 chỉ đạt 0,3780. Khi sửa thành learning rate quá lớn (khoảng $2,6 \times 10^{-3}$), kiến trúc Post-LN lại mất ổn định. Chỉ khi learning rate đỉnh ở khoảng $5,3 \times 10^{-4}$, mô hình mới hội tụ tốt. Thay Post-LN bằng Pre-LN cho kết quả tương đương qua ba seed (0,6029 ± 0,0072 so với 0,6069 ± 0,0080). Điều này cho thấy khi huấn luyện Transformer từ đầu, việc thiết lập đúng lịch học quan trọng không kém việc cài đặt đúng kiến trúc.

Ngoài ra, nhóm đã đóng gói mô hình thành một ứng dụng web minh hoạ gồm backend FastAPI và giao diện React (mục 3.6). Ứng dụng mô phỏng khung bình luận của mạng xã hội: mỗi bình luận được gắn nhãn ngay khi gửi, bình luận độc hại được làm mờ, và người dùng có thể xem chi tiết các bước tiền xử lý cùng trọng số attention của từng câu.

4.2. Hạn chế

- Lớp OFFENSIVE vẫn khó phân biệt (F1 = 0,3871), thường bị nhầm với cả CLEAN lẫn HATE. Precision của HATE còn thấp (0,4546). Mô hình vẫn kém PhoBERT ở cả ba lớp.
- Mô hình được huấn luyện từ trọng số ngẫu nhiên, không có tri thức ngôn ngữ tiền huấn luyện, chỉ học từ khoảng 24 nghìn câu với phần lớn là CLEAN. Mô hình bắt đầu overfitting từ khoảng epoch 7.
- Từ điển mức từ không xử lý được biến thể chính tả, teencode và từ mới. Có 8,4% token trên tập test là `<unk>`.
- Kết quả phụ thuộc vào chất lượng tách từ của underthesea, vốn được huấn luyện trên văn bản chuẩn, chưa tối ưu cho ngôn ngữ mạng xã hội.
- Kết quả macro-F1 chênh lệch khoảng 0,01 giữa các lần chạy (backend MPS không tất định tuyệt đối). Đề tài mới đánh giá ba seed cho hai cấu hình chính và khảo sát ba mức learning rate, chưa khảo sát các siêu tham số khác (số lớp, `d_model`, dropout, MAX_LEN). Vì đã xem kết quả test của nhiều cấu hình trong quá trình khảo sát, con số test có thể hơi lạc quan.
- PhoBERT chỉ được fine-tune một lần với một bộ siêu tham số, nên so sánh ở mục 3.4.5 chỉ mang tính tham chiếu.
- Ứng dụng minh hoạ mới chạy trên máy cục bộ, chưa có xác thực người dùng, giới hạn tần suất truy cập hay cơ chế để người kiểm duyệt sửa nhãn sai, nên chưa sẵn sàng triển khai như một dịch vụ kiểm duyệt thực tế.

4.3. Hướng phát triển

- Tăng số seed (ví dụ từ 5 đến 10 seed) và dùng một tập validation riêng tách từ train để chọn siêu tham số, giữ tập test hoàn toàn độc lập cho lần đánh giá cuối cùng.
- Áp dụng early stopping theo dev macro-F1, tăng dropout hoặc weight decay để giảm overfitting, đồng thời khảo sát thêm số lớp encoder, `d_model` và `MAX_LEN`.
- Khởi tạo lớp embedding bằng word embedding tiếng Việt đã huấn luyện sẵn (ví dụ fastText [17]) thay vì ngẫu nhiên, hoặc tự tiền huấn luyện mô hình bằng masked language modeling trên corpus bình luận không gán nhãn.

- Thay từ điển mức từ bằng tokenizer subword (BPE hoặc WordPiece) để giảm tỉ lệ <unk> và xử lý tốt hơn các biến thể chính tả, teencode.
- Thử focal loss [18] hoặc tăng cường dữ liệu cho các lớp thiểu số để cải thiện cân bằng giữa precision và recall của OFFENSIVE và HATE.
- Từ công cụ trực quan hoá attention trong ứng dụng (mục 3.6), phân tích có hệ thống trên nhiều bình luận thật để tìm hiểu mô hình "chú ý" vào những từ nào khi ra quyết định, và đối chiếu với các phương pháp giải thích mô hình khác.
- Triển khai ứng dụng lên máy chủ và bổ sung chức năng cho người kiểm duyệt đánh dấu nhãn sai, nhằm thu thập thêm dữ liệu gán nhãn để huấn luyện lại mô hình.
- Thu hẹp khoảng cách với PhoBERT [3] theo hướng tiền huấn luyện nhẹ: pretrain chính mô hình nhỏ này bằng masked language modeling trên corpus bình luận tiếng Việt, rồi fine-tune trên ViHSD.

TÀI LIỆU THAM KHẢO

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention Is All You Need," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.

[2] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," in *Proceedings of NAACL-HLT*, pp. 4171–4186, 2019.

[3] D. Q. Nguyen and A. T. Nguyen, "PhoBERT: Pre-trained language models for Vietnamese," in *Findings of the Association for Computational Linguistics: EMNLP 2020*, pp. 1037–1042, 2020.

[4] S. T. Luu, K. V. Nguyen, and N. L.-T. Nguyen, "A Large-scale Dataset for Hate Speech Detection on Vietnamese Social Media Texts," in *Advances and Trends in Artificial Intelligence. Artificial Intelligence Practices (IEA/AIE 2021)*, Lecture Notes in Computer Science, vol. 12798, Springer, 2021.

[5] A. Schmidt and M. Wiegand, "A Survey on Hate Speech Detection using Natural Language Processing," in *Proceedings of the Fifth International Workshop on Natural Language Processing for Social Media (SocialNLP)*, pp. 1–10, 2017.

[6] T. Davidson, D. Warmsley, M. Macy, and I. Weber, "Automated Hate Speech Detection and the Problem of Offensive Language," in *Proceedings of the International AAAI Conference on Web and Social Media (ICWSM)*, vol. 11, no. 1, 2017.

[7] S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[8] Y. Kim, "Convolutional Neural Networks for Sentence Classification," in *Proceedings of EMNLP*, pp. 1746–1751, 2014.

[9] D. Hendrycks and K. Gimpel, "Gaussian Error Linear Units (GELUs)," *arXiv preprint arXiv:1606.08415*, 2016.

[10] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in *Proceedings of IEEE CVPR*, pp. 770–778, 2016.

[11] J. L. Ba, J. R. Kiros, and G. E. Hinton, "Layer Normalization," *arXiv preprint arXiv:1607.06450*, 2016.

[12] R. Xiong, Y. Yang, D. He, K. Zheng, S. Zheng, C. Xing, H. Zhang, Y. Lan, L. Wang, and T.-Y. Liu, "On Layer Normalization in the Transformer Architecture," in *Proceedings of ICML*, pp. 10524–10533, 2020.

[13] D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," in *International Conference on Learning Representations (ICLR)*, 2015.

[14] R. Pascanu, T. Mikolov, and Y. Bengio, "On the difficulty of training Recurrent Neural Networks," in *Proceedings of ICML*, pp. 1310–1318, 2013.

[15] Underthesea: Vietnamese NLP Toolkit, https://github.com/undertheseanlp/underthesea

[16] A. Paszke et al., "PyTorch: An Imperative Style, High-Performance Deep Learning Library," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019.

[17] P. Bojanowski, E. Grave, A. Joulin, and T. Mikolov, "Enriching Word Vectors with Subword Information," *Transactions of the Association for Computational Linguistics*, vol. 5, pp. 135–146, 2017.

[18] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal Loss for Dense Object Detection," in *Proceedings of IEEE ICCV*, pp. 2980–2988, 2017.